Internal Developer Platform

The complete project guide: what it does, how every part works, why it was built this way, and how to explain it.

1. The project in one minute
2. The big picture
3. What happens, step by step
4. Backend
5. Frontend
6. The Helm chart
7. GitOps and ArgoCD
8. Login and security
9. Networking and DNS
10. Terraform
11. CI/CD pipeline
12. Design decisions
13. Limits and costs
14. Everyday commands
15. Questions you'll be asked
16. Glossary

---

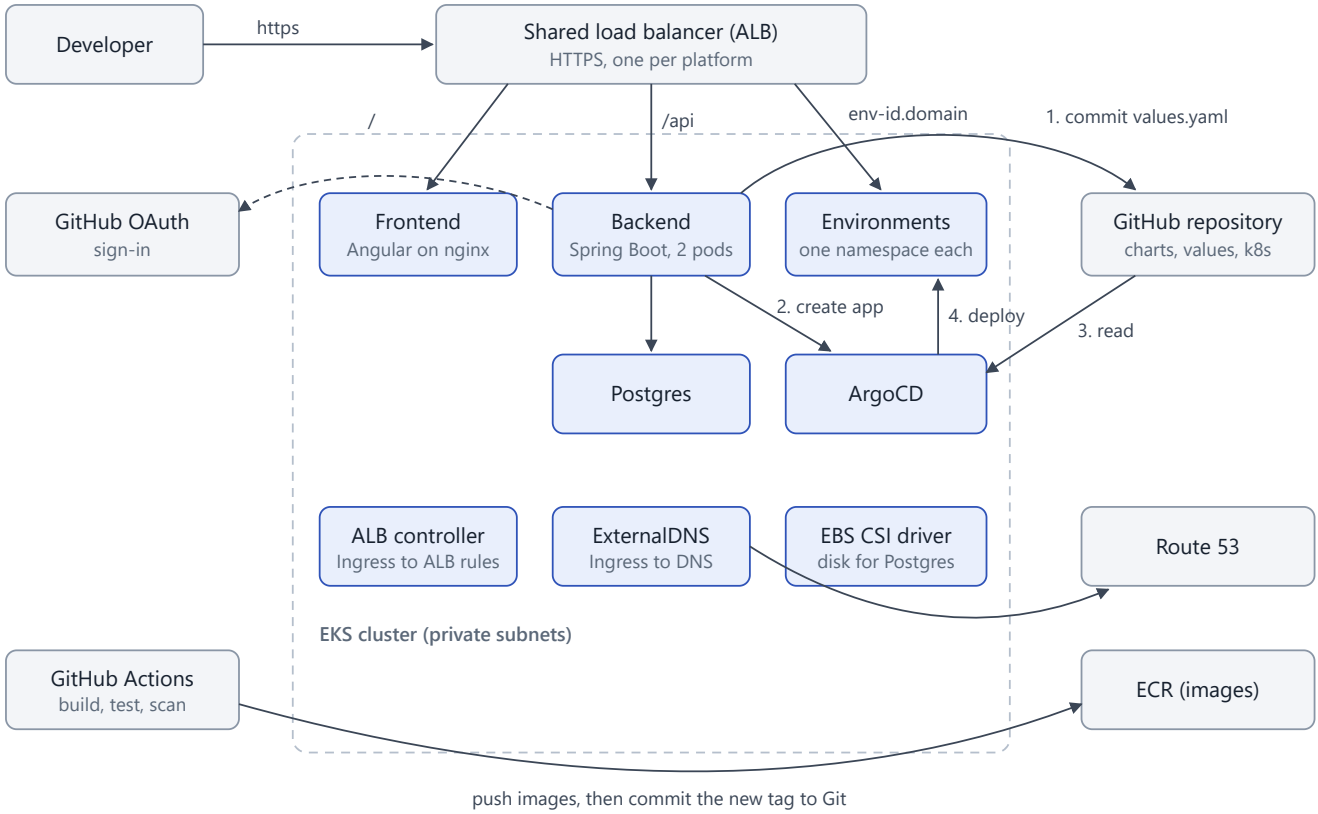
1. The project in one minute

Normally, a developer who needs a test environment opens a ticket and waits for someone from DevOps. This platform replaces that with a form. The developer picks a team, a type and the container images to run, and about two minutes later has a live environment at its own HTTPS address.

Three ideas make it more than a form that calls Kubernetes:

- **Git is the source of truth.** The portal doesn't deploy anything. It writes a file to Git, and ArgoCD makes the cluster match. That's GitOps: every environment that ever existed is in the Git history, and any manual change in the cluster gets reverted.
- **Environments are temporary.** Each has a lifetime of 1 to 72 hours, and a background job deletes it when time runs out. Forgotten environments are what make cloud bills grow.
- **The whole platform is code.** Terraform creates all the AWS infrastructure, and the platform deploys itself through the same GitOps flow it offers users. It can be destroyed and rebuilt from one repository, and it was, repeatedly, to save money.

2. The big picture



Numbers show the order when creating an environment. Terraform creates everything here except the GitHub services and the pods inside the cluster.

What lives where

Folder	What it is
`idp-backend/`	Spring Boot 4 API, Java 21
`idp-frontend/`	Angular 19 portal. `Dockerfile` runs the dev server (local use), `Dockerfile.prod` builds for nginx (AWS).
`charts/environment/`	Helm chart rendered once per environment
`environments/<id>/`	One `values.yaml` per environment, written by the backend. Never edit these by hand.
`k8s/`	The platform's own manifests (backend, frontend, Postgres, Ingress). ArgoCD deploys this folder.
`argocd/platfrom-app.yml`	The ArgoCD Application that points at `k8s/`. (The typo in the file name is real, and the commands use it.)
`terraform/`	All AWS infrastructure
`.github/workflows/ci-cd.yml`	The pipeline
`demo/`	Scripts that recorded the demo video (Playwright and ffmpeg)

3. What happens, step by step

Creating an environment

1. **Form.** Angular validates the input and posts to `/api/environment/create`, with the session cookie and CSRF header.
2. **ID.** `EnvironmentService.generateEnvId` builds something like `dev-payments-mw1abc`: type, team, then the current time in base 36 so it's unique. Anything outside `a-z 0-9 -` becomes a dash, because this ID becomes a Kubernetes namespace.
3. **Values.** `TemplateService` turns the request into YAML with SnakeYAML: the subdomain plus the list of services.
4. **Git.** `GitOpsService` clones the `dev` branch with JGit into a temp folder, writes `environments/<id>/values.yaml`, commits and pushes.
5. **ArgoCD.** `ArgoCdService` POSTs an Application to ArgoCD's REST API: use the chart at `charts/environment`, with that values file, in a namespace named after the ID, syncing automatically.
6. **Database.** The environment and its services are saved in Postgres, with `expiresAt` set to now plus the TTL.
7. **Cluster.** ArgoCD renders the chart and applies it. Kubernetes starts the pods.
8. **Load balancer.** The AWS Load Balancer Controller sees the new Ingress and adds a rule for its host to the shared ALB. Because the ALB already exists, this takes seconds. A new ALB would take 2 to 3 minutes.
9. **DNS.** ExternalDNS sees the Ingress and creates the Route 53 record.
10. **Status.** The detail page polls `/status` every 5 seconds until ArgoCD says Healthy or Degraded, then shows the URL.

When a create fails

Git and ArgoCD can't take part in a database transaction, so `createEnvironment` rolls back by hand. If the ArgoCD call or the database save throws, it deletes the ArgoCD app and the Git folder, then reports the original error. Each undo step runs even if the other fails. Without this, you'd get an environment running in the cluster that the portal doesn't know about, so nobody could see it or destroy it.

Destroying an environment

1. Delete the ArgoCD Application with `cascade=true`, which removes its workloads from the cluster. A 404 counts as success.
2. Delete the folder from Git. If it's already gone, nothing happens.
3. Delete the database row, last on purpose. If step 1 or 2 fails, the row survives and the destroy can be repeated.

The controller then removes the environment's ALB rules, and ExternalDNS removes its DNS record. The empty namespace stays behind, because ArgoCD doesn't delete namespaces it created with `CreateNamespace=true`. It costs nothing, and goes away with the cluster.

Expiry

`CleanupService.destroyExpired` runs every 5 minutes (`@Scheduled`), finds rows with `expiresAt` in the past and calls the same `destroy` as the button. Each destroy has its own try/catch, so one failure doesn't block the others, and the surviving row gets retried next time. Both backend replicas run this job. That's safe because every destroy step can be repeated, so the worst case is one replica logging a harmless error.

Deploying a change to the platform

1. You push to `dev`. GitHub Actions builds, tests and scans.
2. It builds both Docker images and pushes them to ECR, tagged with the commit SHA.
3. It edits the image tag in `k8s/backend/deployment.yml` and `k8s/frontend/deployment.yml` using `yq`, and commits that with `[skip ci]` so it doesn't trigger itself.
4. ArgoCD sees the change (it checks every 3 minutes) and does a rolling update.

CI has no access to the cluster at all. It only has rights to ECR and to the repository. That's the "pull model" of GitOps.

---

4. Backend

Spring Boot 4.1 with Java 21. Note that Boot 4 uses **Jackson 3**: Jackson 2 types such as `com.fasterxml` `JsonNode` are still on the classpath through fabric8 but fail at runtime, which is why `ArgoCdService` parses ArgoCD's replies as plain Maps.

Class	Job
`EnvironmentController`	REST endpoints under `/api/environment`
`AuthController`	`GET /api/me`. Also reads the CSRF token, which is what makes Spring set the XSRF cookie for Angular.
`EnvironmentService`	The main logic: create with rollback, TTL limits, list, status, destroy
`TemplateService`	Request to `values.yaml`
`GitOpsService`	Clone, change, commit, push with JGit. Checks each push's status and retries from a fresh clone if rejected, up to 3 times. It needs this because JGit doesn't throw when a push is rejected.
`ArgoCdService`	Create, delete and read Applications through ArgoCD's REST API
`CleanupService`	The expiry job
`SecurityConfig`	Who can reach what: login, CSRF, logout, 401 responses
`AllowedUsersService`	Rejects GitHub accounts that aren't on the allowed list

Data model

Table	Columns
`environments`	id, env_id, team, type, status, url, created_by, created_at, last_active_at, expires_at
`environment_services`	id, service_name, port, path, image, health_path, environment_id
`environment_service_env`	service_id, env_name, env_value
`spring_session*`	Login sessions, managed by Spring Session

Hibernate creates and updates these on startup (`ddl-auto=update`). That's convenient for a project this size. A larger production system would use Flyway or Liquibase migrations.

Configuration

Variable	Purpose
`GIT_PASSWORD`	GitHub token with Contents write on the repository
`ARGOCD_URL`, `ARGOCD_TOKEN`	Where ArgoCD is, and the backend's never-expiring key
`GITHUB_CLIENT_ID`, `GITHUB_CLIENT_SECRET`	The OAuth App. One app for local, one for production, since GitHub allows only one callback per app.
`IDP_ALLOWED_USERS`	Comma-separated GitHub usernames allowed to sign in
`IDP_DOMAIN`	Base domain for environments. Default `idplatform.tech`.
`SPRING_DATASOURCE_*`	Database connection

In the cluster, non-secret values come from the ConfigMap `app-config` and secret ones from the Secret `app-secret`. To run the backend from a terminal, export the variables first, because `mvnw spring-boot:run` does not pick up `.env`:

```
cd idp-backend
set -a; . <(tr -d '\r' < .env); set +a
JAVA_HOME="/c/Program Files/Java/jdk-21.0.10" ./mvnw spring-boot:run
```

5. Frontend

Angular 19 with standalone components and signals, lazy-loaded routes and PrimeNG's theme. In AWS it's a static build served by nginx. Locally it's `ng serve`, with `proxy.conf.json` forwarding `/api` to the backend.

File	Job
`core/auth.service.ts`	Holds the signed-in user in a signal, and handles login and logout
`core/auth.interceptor.ts`	Any 401 drops the user back to the sign-in screen
`core/environment.service.ts`	API calls
`pages/environments`	List with live status for each environment
`pages/environment-create`	The form, using nested `FormArray`s: services, and variables inside each service
`pages/environment-detail`	Details, status polling, URL, destroy with typed confirmation

`nginx.conf` returns `index.html` for unknown paths. Routes like `/environments/abc` exist only in the browser, so without this a page refresh would give a 404. The dev proxy has `changeOrigin: false` so the backend sees `localhost:4200` as the host and builds the correct GitHub callback URL.

6. The Helm chart

`charts/environment` loops over `.Values.services` and renders a Deployment and Service per service, plus one Ingress with a path per service. This is what the backend writes for one environment:

```
ingress:
  host: dev-payments-mw1abc.idplatform.tech
services:
  - serviceName: api
    image: nginx:alpine
    port: 80
    path: /
    healthPath: /
    env:
      LOG_LEVEL: info
```

The chart's own `values.yaml` supplies the rest: 1 replica, the ALB group name and the certificate ARN. Test it without a cluster:

```
helm template test charts/environment -f charts/environment/ci/test-values.yaml
```

The chart replaced an earlier FreeMarker version that generated raw manifests. Now the templates live in one versioned place and each environment contributes only its data.

---

7. GitOps and ArgoCD

There are two kinds of ArgoCD Application:

- `idp-platform` deploys the portal itself from `k8s/`. It's applied by hand once, per cluster.
- **One per environment**, created by the backend through the API. Each uses the Helm chart with its own values file.

Both track the `dev` branch with **automated sync**, **prune** (things removed from Git get removed from the cluster) and **selfHeal** (manual changes in the cluster get reverted). That's why deleting a Deployment by hand doesn't stick: ArgoCD puts it back.

The backend logs in as a dedicated account, `idp-backend`, defined in `terraform/argocd.tf`. Its API key has no expiry, and its rights cover Applications only. It can't, for example, add repositories.

---

8. Login and security

How login works

1. Angular calls `/api/me`. A 401 means nobody is signed in, so it shows the sign-in screen.
2. The button navigates to `/api/oauth2/authorization/github`. Spring redirects the browser to GitHub.
3. The user approves. GitHub redirects back to `/api/login/oauth2/code/github` with a one-time code.
4. Spring swaps the code for an access token and fetches the profile.
5. `AllowedUsersService` checks the username. If it isn't listed, the user lands on `/?denied`.
6. Spring creates a session and sets a `SESSION` cookie.

Compared with JWT

JWT	This project (server-side session)
The token holds the user's data and is signed	The cookie holds only an ID. The data is in Postgres.

Hard to revoke before it expires	Delete the row and the user is signed out
JavaScript has to store and send it	The browser sends it automatically, and JavaScript can't read it
No CSRF risk when sent in a header	Needs CSRF protection, because cookies are sent automatically

Sessions are in Postgres because there are 2 backend replicas. A session kept in one pod's memory would be lost as soon as the load balancer sent the next request to the other pod.

CSRF

Spring sets a readable `XSRF-TOKEN` cookie. Angular's HttpClient copies it into an `X-XSRF-TOKEN` header on POST and DELETE. Another website can make the browser send cookies but can't read them, so it can't produce that header.

Permissions in AWS

Controllers that need AWS access use **IRSA**: an IAM role tied to one Kubernetes service account, with temporary credentials and no stored keys. There are three: `idp-alb-controller`, `idp-ebs-csi` and `idp-external-dns`. ExternalDNS can only change this one hosted zone.

Secrets

`k8s/secret.yml`, `k8s/database/secret.yml` and `.env` are gitignored and applied by hand. ArgoCD deploys everything in `k8s/` from Git, and since these aren't in Git, it never sees them. A step up would be Sealed Secrets or External Secrets.

---

9. Networking and DNS

The network: one VPC with public and private subnets in 2 availability zones. Nodes sit in private subnets, and the load balancer in public ones. Outbound traffic from nodes goes through one NAT gateway, which is cheaper than one per zone but is a single point of failure.

The path of a request: browser → Route 53 → ALB (HTTPS with the ACM certificate) → directly to the pod's IP (`target-type: ip`), skipping NodePorts.

One shared ALB. Each Ingress normally creates its own ALB, at about $16 to $18 a month each. Giving them all the annotation `alb.ingress.kubernetes.io/group.name: idp-shared` merges them into one, and each Ingress only adds rules. Settings like scheme, ports and redirect must match across the group, so `k8s/ingress.yml` and the chart use the same values.

Rule order matters. In the portal's Ingress, `/api` is listed before `/`, because `/` matches everything.

Health checks. Each Service sets its own check path with an annotation. The backend uses `/actuator/health`, which is public. Redis is excluded from that check, because the cluster has no Redis host and it would report DOWN (503). If all targets are unhealthy, an ALB "fails open" and sends traffic to them anyway, so a broken health check can look like it's working.

The certificate. One ACM certificate covers `idplatform.tech` and `*.idplatform.tech`. It's kept outside Terraform so it survives a destroy.

ExternalDNS. It watches Ingresses and creates or removes records, with a TXT record marking each one as its own. It never touches records it didn't create. That's why a hand-made record has to be deleted once before it can take over that name.

10. Terraform

File	Creates
`main.tf`	AWS and Helm providers. Helm fetches a cluster token at apply time, since the cluster doesn't exist at plan time.
`vpc.tf`	VPC 10.0.0.0/16, 2 public and 2 private subnets, 1 NAT gateway, and the subnet tags the ALB controller looks for
`eks.tf`	EKS 1.34, 3 t3.small nodes on Amazon Linux 2023, IRSA, EBS CSI add-on
`ebs-csi.tf`	IAM role for the disk driver. Without the driver, Postgres's volume claim stays Pending forever.
`alb-controller.tf`	IAM role and Helm release that turns Ingresses into ALB rules
`external-dns.tf`	IAM role and Helm release for ExternalDNS, plus a lookup of the existing hosted zone
`argocd.tf`	ArgoCD Helm release, in plain HTTP mode inside the cluster, plus the `idp-backend` account
`ecr.tf`	Image repositories for frontend and backend. `force_delete` lets a destroy remove them with images inside.
`elasticache.tf`, `security-groups.tf`	A Redis node and its firewall rule. Nothing uses it yet.
`iam.tf`	A read-only role left from an earlier design. Nothing uses it.

Terraform stops at the platform's foundations. It installs ArgoCD, and ArgoCD handles everything above that. State is local in `terraform/terraform.tfstate`, which is gitignored and contains secrets.

11. CI/CD pipeline

Job	What it does
Build	`mvn install` on Java 21
Test	Starts a real Postgres and runs `mvn test`. `contextLoads` boots the whole app, so it catches wiring and config errors. Also runs Checkstyle.
SecurityScan	Trivy scans the repository. It's set to `exit-code: 0`, so it reports problems but doesn't fail the build.
Build and push	Only on `dev`, after the others pass. Builds images, pushes to ECR, commits the new tags, and retries the push if the portal pushed at the same moment.

`paths-ignore` skips the pipeline for changes in `k8s/` and `environments/`, so the portal's commits don't trigger builds.

Repository settings it relies on: secrets `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`, and the variable `AWS_REGION`, in the `IDP` environment.

12. Design decisions

Decision	Why	What it gives up
GitOps, not direct kubectl from the backend	History, easy rollback, the cluster repairs itself, and the backend needs no cluster credentials	Slower, and Git becomes a dependency of every create
Helm chart, not generated manifests	One template, and each environment is just data	Needs Helm knowledge
One namespace per environment	Isolation, and cleanup is simple	Many namespaces to look through
Shared ALB with an Ingress group	About $16 to $18 a month saved per environment, and faster to become reachable	All environments share one failure point and a limit of about 100 rules
Server-side sessions in Postgres	Works with 2 replicas, and can be revoked instantly	A database read per request, and needs CSRF protection
GitHub login with an allowed list	No passwords to store, and teams already have GitHub accounts	Depends on GitHub, and the list lives in a ConfigMap
Postgres in the cluster, not RDS	Much cheaper for a demo-sized project	Data is lost on every destroy, and there are no managed backups
Nginx frontend image	Far lighter than the dev server, about 32 MB instead of 512 MB requested	Separate Dockerfiles for local and production
Rollback by hand, not a saga framework	Three steps don't justify a framework	A crash in the middle of a create can still leave leftovers

13. Limits and costs

What it doesn't do

- **Secrets:** environment variables are stored in plain text in a public repository. Don't put passwords in them. The form warns about this.
- **Quotas:** nothing limits how many environments a team creates.
- **Cluster size:** 3 t3.small nodes allow 33 pods in total, and the platform itself uses about 28. Only about **4 or 5 environment services fit at once**. Beyond that, pods stay Pending. Change `instance_types` or `desired_size` in `eks.tf` for more, within the account's limit of 8 vCPUs.
- **Editing:** an environment can't be changed, only destroyed and recreated.
- **Environment database:** deployed apps can't request a database.
- **Monitoring:** only ArgoCD's health status. No metrics, log search or alerts.
- **Terraform state:** local, so only one person can safely run it.
- **Debug port:** the backend image starts with a debug agent on 5005. No Service exposes it, but it should be removed from the production image.

- **Unused resources:** Redis costs about $13 a month while running and nothing uses it. Remove `elasticache.tf`, `security-groups.tf` and its output when you want to save that.
- **Old folders:** `environments/` holds values from before destroy started cleaning up. They do nothing and can be deleted.

Rough running costs

Item	Per hour (approximate)
EKS control plane	$0.10
3 t3.small nodes	$0.07
NAT gateway	$0.05, plus data
Load balancer	$0.03
Redis, disks, DNS	$0.03
Total	**About $0.30, or $7 a day**

---

14. Everyday commands

```
# See everything
kubectl get pods -A
kubectl get applications -n argocd

# Logs
kubectl logs -n idp-project deploy/idp-backend-deployment --tail=50
kubectl logs -n kube-system deploy/external-dns --tail=20
kubectl logs -n kube-system deploy/aws-load-balancer-controller --tail=30

# Why is this environment unhealthy?
kubectl get pods,ingress -n <env-id>
kubectl describe pod -n <env-id> <pod-name>

# Open the ArgoCD dashboard at http://localhost:8089 (user admin)
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d; echo
kubectl port-forward svc/argocd-server -n argocd 8089:80

# Restart the backend after changing a secret
kubectl apply -f k8s/secret.yml
kubectl rollout restart deploy/idp-backend-deployment -n idp-project

# Is the load balancer happy with a service's pods?
aws elbv2 describe-target-groups --region eu-north-1 \
  --query 'TargetGroups[].[TargetGroupName,HealthCheckPath]' --output text
```

Common changes

To change	Edit
Who can sign in	`IDP_ALLOWED_USERS` in `k8s/configmap.yml`, push, then restart the backend

TTL limits or default	The constants at the top of `EnvironmentService.java`, and the options in the create form
What each environment pod gets	`charts/environment/templates/deployment.yml`. Applies through Git on the next sync.
Node size or count	`terraform/eks.tf`, then plan and apply
Domain	`variable "domain"` in Terraform, `IDP_DOMAIN`, the host in `k8s/ingress.yml`, the certificate, and the OAuth App's callback

15. Questions you'll be asked

Why GitOps instead of having the backend call Kubernetes?

Git gives a full history of every environment, a simple way to roll back, and self-repair: ArgoCD reverts drift. The backend also never holds cluster credentials, so a compromised backend can't run arbitrary things on the cluster directly. The trade-off is that creates depend on Git and are slower.

What happens if ArgoCD goes down?

Running environments keep running, since ArgoCD only reconciles. New creates fail at the ArgoCD call and roll back their Git commit. When ArgoCD returns, it catches up from Git.

What if the backend crashes in the middle of a create?

The rollback code only runs if the process is alive, so a crash between steps can leave a Git folder or an ArgoCD app with no database row. A fuller design would write the intent to the database first and have a job reconcile it. The TTL job doesn't catch this today, since it works from database rows.

Why do both replicas run the cleanup job, and isn't that a problem?

Every step of destroy can safely be repeated, so running twice causes at most a logged error. For heavier jobs I'd add a lock such as ShedLock.

Why sessions and not JWT?

The browser is the only client, sessions can be revoked at once, and Spring Security provides them out of the box. The cost is CSRF protection and a shared store, so Postgres holds them for both replicas.

How do environments stay isolated from each other?

Separate namespaces, which separate names and resources. They aren't a security boundary without NetworkPolicies, so pods can still reach each other across namespaces. I'd add default-deny NetworkPolicies and ResourceQuotas next.

How would you scale this to many teams?

Quotas per team, cluster autoscaling (Karpenter), remote Terraform state with locking, secrets from a vault, and monitoring. Git pushes become a bottleneck under heavy use, so I'd queue them, and perhaps give environment values their own repository.

What was the hardest part?

Things that fail quietly. JGit doesn't throw on a rejected push. An ALB fails open, so a broken health check looks fine. `--legacy-peer-deps` hid a missing dependency until the first production build. Each was found by running against real infrastructure, not by unit tests.

How much does it cost, and how did you control that?

About $7 a day while running. Environments expire automatically, they share one load balancer, Postgres runs in the cluster instead of RDS, and the whole stack can be destroyed and rebuilt from code when it's not needed.

---

16. Glossary

ACM	AWS Certificate Manager, free HTTPS certificates
ALB	Application Load Balancer, routes HTTP(S) by host and path
ArgoCD	Runs in the cluster and keeps it matching what's in Git
CSRF	An attack where another site makes your browser send a request with your cookies
ECR	AWS's Docker image registry
EKS	AWS's managed Kubernetes
ExternalDNS	Keeps DNS records in step with Kubernetes Ingresses
GitOps	Git holds the desired state, and an agent makes reality match it
Helm	Templates for Kubernetes manifests
Ingress	A Kubernetes rule for routing outside traffic to Services
IRSA	IAM Roles for Service Accounts, AWS rights for one pod identity without stored keys
NAT gateway	Lets private subnets reach the internet without being reachable from it
OAuth2	Lets a user sign in through GitHub without giving this app their password
Route 53	AWS's DNS
StatefulSet	Like a Deployment, for pods with stable identity and disks
TTL	Time to live: how long an environment exists before it's destroyed

Related documents: `docs/runbook-apply.pdf` and `docs/runbook-destroy.pdf` have the exact commands for bringing the platform up and tearing it down.